

실무 자료구조와 저장 엔진

시간 복잡도만 보면 놓치는 것들

교과서와 실측이 반대로 나온다

$O(1)$ vs $O(n)$

교과서

LinkedList 중간 삽입은 링크 두 개만 바꾸면 끝이다.
ArrayList는 뒤 원소를 전부 밀어야 한다.
그러니 중간 삽입이 잦으면 LinkedList를 쓰라고 배운다.

18배

실측: 10만 개, 가운데 삽입+삭제

ArrayList 12.1 μ s, LinkedList 217 μ s.
크기를 1천, 1만, 10만으로 바꿔도 매번 ArrayList가 이긴다.

교과서가 틀린 걸까?

답은 첫 번째 주제인 캐시 지역성에서 드리겠습니다. 오늘은 이런 '빅오 밖의 비용'을 자료구조 여섯 개로 따라갑니다.

메모리에서 디스크로, 여섯 개의 자료구조

1부 · 메모리 안의 자료구조

1 캐시 지역성

같은 $O(n)$ 인데 왜 18배 차이가 날까

실측

2 해시테이블

평균 $O(1)$ 을 지키는 리사이징과 트리화

3 블룸 필터

없는 키를 적은 메모리로 걸러 DB를 지킨다

실측

4 스킵리스트

Redis 랭킹은 왜 트리 대신 이걸 쓸까

2부 · 디스크 위의 저장 엔진

5 B-tree vs LSM

디스크 위에서 읽기와 쓰기 중 무엇을 내주나

6 WAL

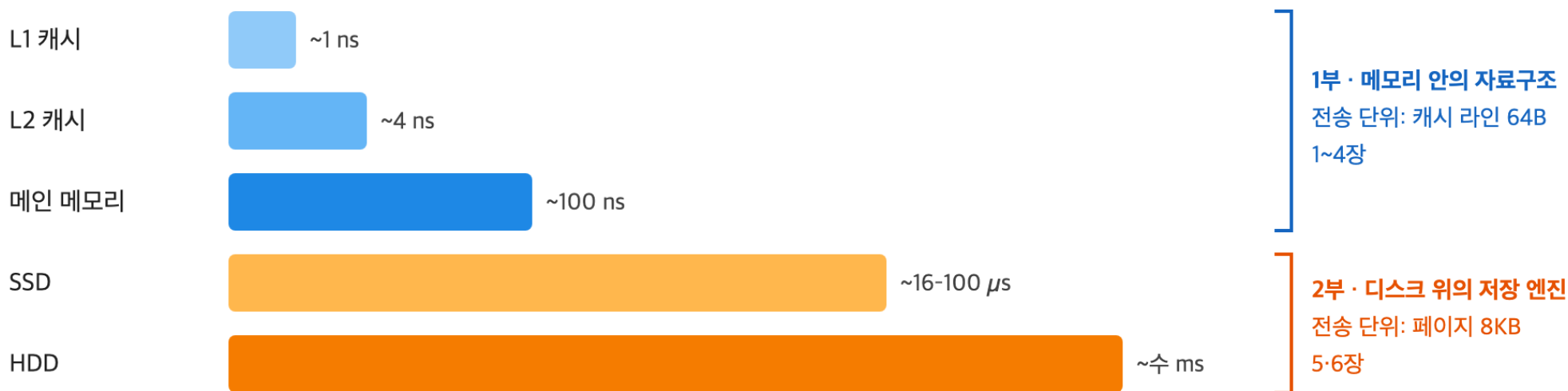
메모리에 든 변경을 크래시에서 지킨다

각 주제는 앞 주제가 남긴 질문에서 출발합니다.

실측 두 개는 Docker 안에서 직접 재현했습니다 (JMH, Redis + PostgreSQL).

같은 '읽기 1번'도 데이터가 어디 있느냐에 따라 다르다

같은 '읽기 1번'의 비용 — 데이터가 어디 있느냐에 따라 천만 배 차이 (로그 스케일)



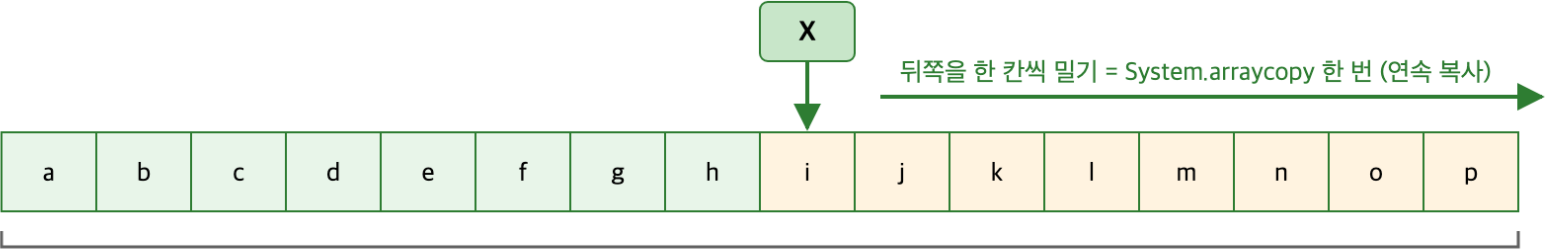
빅오는 이 다섯 줄을 모두 '1번'으로 센다. 실제 비용은 어느 줄까지 내려가느냐와 몇 번 내려가느냐로 정해진다.

수치는 하드웨어마다 다르며 자릿수만 의미가 있다.

CPU는 64바이트(캐시 라인)씩 묶어서 가져오고, 순서대로 읽으면 다음 줄을 미리 가져온다(프리페치). 둘 다 데이터가 붙어 있을 때만 통한다.

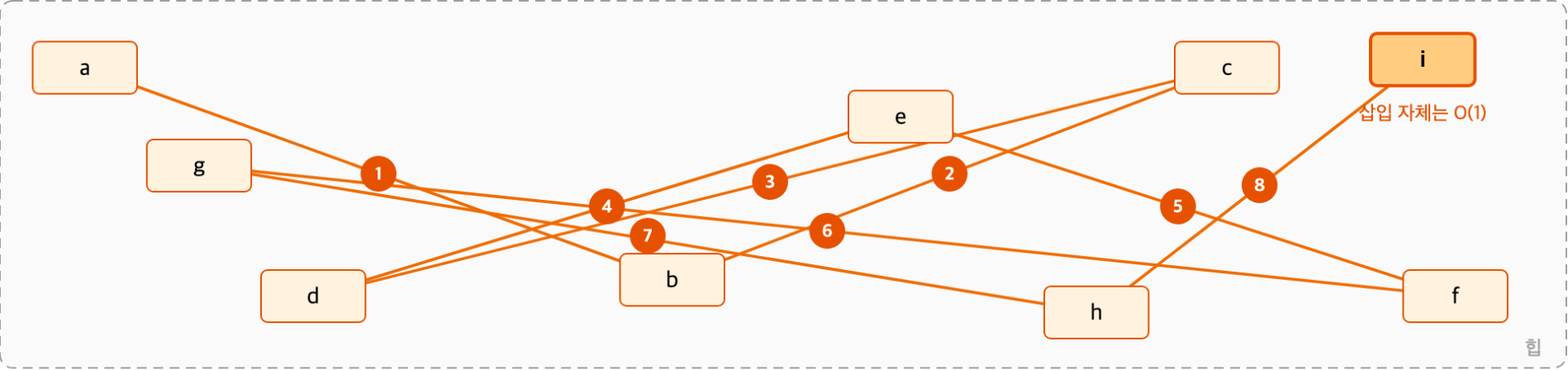
같은 O(n)이어도: 한꺼번에 복사 vs 한 걸음씩 기다리기

ArrayList.add(8, X) — 참조가 배열 하나에 연속으로 놓여 있다



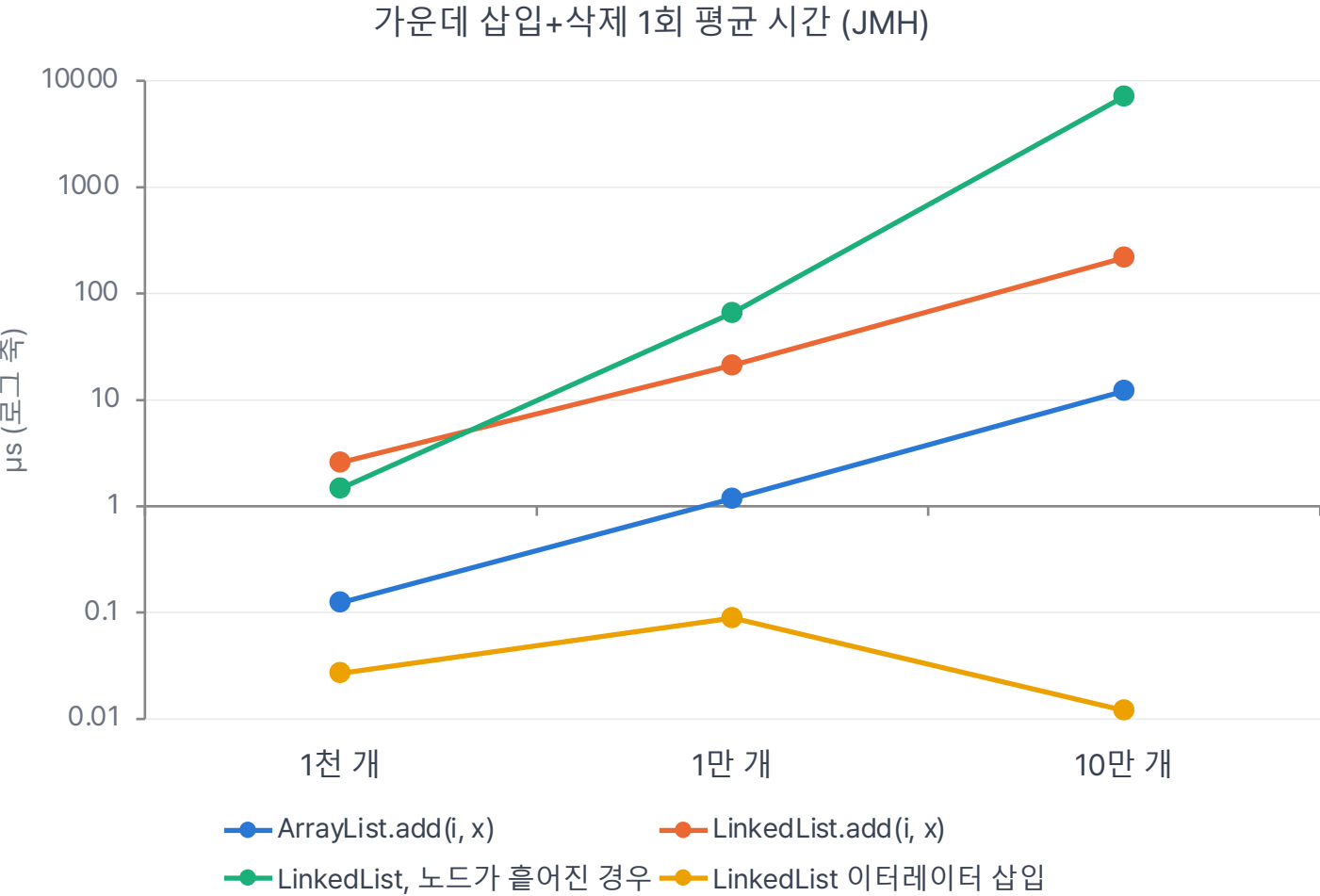
캐시 라인 1개(64B)에 참조(4B) 16개가 들어간다. 메모리 왕복 1번에 전부 도착하고, 다음 라인은 프리페처가 미리 가져온다

LinkedList.add(8, X) — 노드가 힙 곳곳에 흩어져 있다



삽입할 자리(i)에 닿기 전에 포인터를 8번 따라가야 한다. 다음 주소는 지금 노드를 읽어야 알 수 있어 한 걸음씩 기다린다.
노드가 이렇게 흩어져 있으면 번호 하나하나가 **메인 메모리 왕복(~100 ns)**이다. ArrayList는 같은 O(n)을 연속 복사 한 번으로 끝낸다.

실측: 캐시 안에서도 18배, 캐시 밖이면 수백 배



18배

캐시 안에서도 ArrayList가 빠르다
노드 한 걸음 약 2ns. 캐시에 맞았지만 다음 주소를 기다리는 사슬이 남는다

ms 단위

노드가 흩어지면 수백 배
한 걸음 약 70ns, 메인 메모리 왕복 수준. 같은 LinkedList, 같은 O(n)

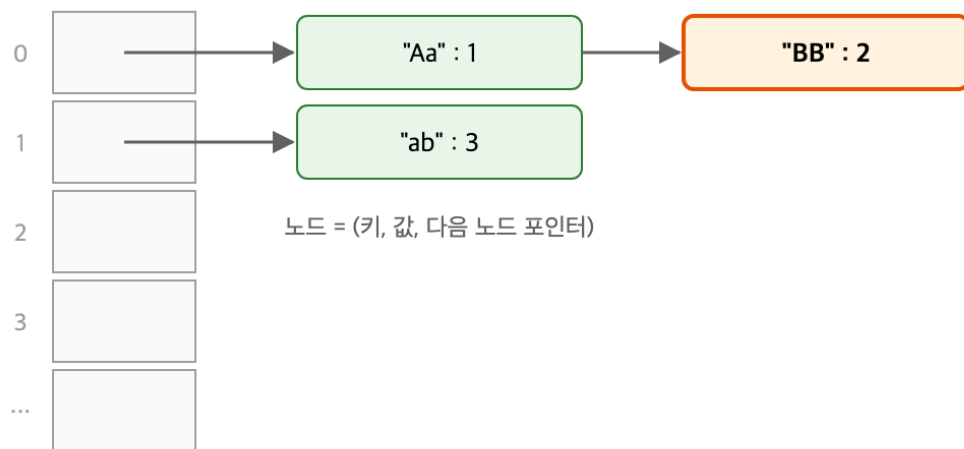
12ns

위치를 쥐고 있으면 O(1)은 진짜
이터레이터 삽입은 크기와 무관하다

충돌하면 같은 칸에 줄을 선다: 조회 비용 = 줄 길이

충돌과 체이닝: 같은 칸에 온 원소는 리스트로 줄을 세운다

"Aa".hashCode() = "BB".hashCode() = 2112, 둘 다 0번 칸



get("BB")가 실제로 하는 일

- ① "BB"의 해시를 계산해서 0번 칸으로 간다
- ② 첫 노드 "Aa"와 비교: equals? **아니오**
- ③ 다음 노드 "BB"와 비교: equals? **예**
- ④ 값 2를 돌려준다

해시가 같아도 키는 다를 수 있으니
마지막 확인은 반드시 equals로 한다

칸 안의 줄이 짧으면 거의 바로 찾는다. 줄이 길어질수록 ②③ 같은 비교가 늘어난다. 해시테이블의 속도는 결국 **줄의 길이**이다.

적재율 = 원소 수 ÷ 칸 수

줄의 평균 길이. 평균 O(1)은 이 값이 작을 때만 성립한다

조건 ①: 칸이 충분할 것

리사이징이 지킨다 (다음 장)

조건 ②: 키가 고르게 퍼질 것

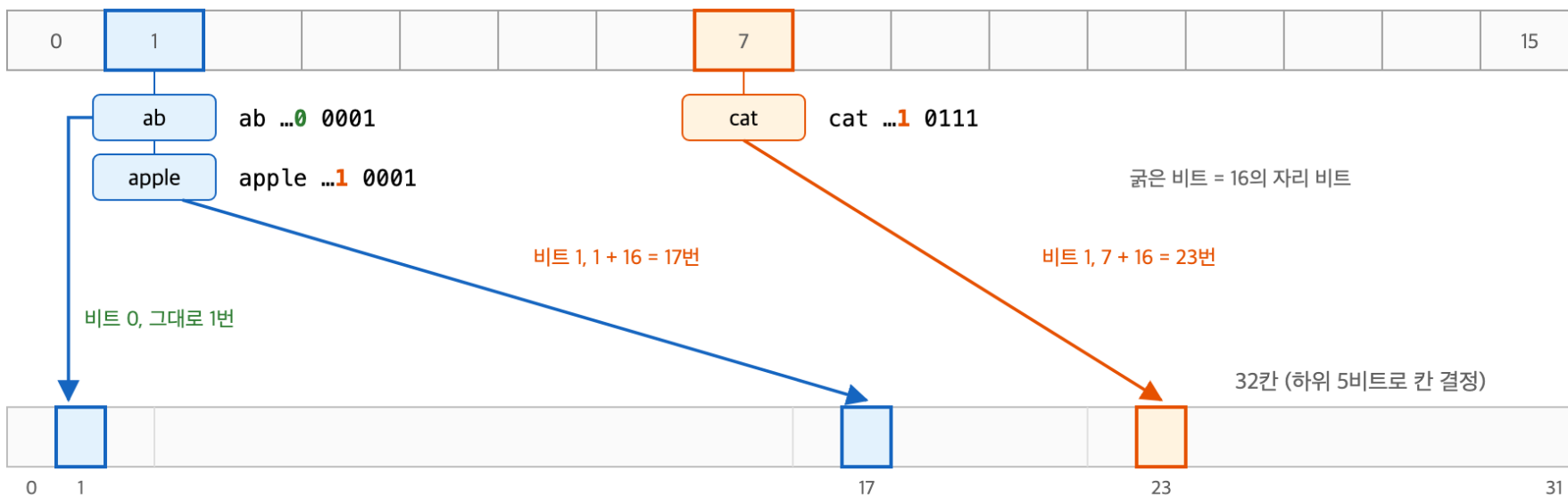
깨지면 트리화가 최악을 막는다 (그다음 장)

앞에서: 흩어진 노드를 따라가는 비용 → 충돌 체인도 결국 작은 *LinkedList*다. 그래서 줄을 짧게 지켜야 한다

리사이징: 0.75를 넘으면 2배, 원소는 제자리 또는 +16

리사이징: 16칸에서 32칸으로. 각 원소는 '제자리' 아니면 '제자리 + 16'으로만 간다

16칸 (하위 4비트로 칸 결정)



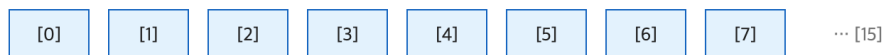
칸 수가 2배가 되면 칸 번호를 정하는 비트가 하나 늘어난다. 늘어난 비트가 0이면 그대로, 1이면 +16.

그래서 해시를 처음부터 다시 계산하지 않고, 칸마다 줄을 '그대로 팀'과 '+16 팀' 둘로 나누기만 하면 된다.

평균은 $O(1)$ (분할 상황), 그 한 번의 put은 $O(n)$. 원소 수를 알면 처음부터 크게 만든다: `HashMap.newHashMap(n)` (JDK 19+)

한 칸에 몰리면: 그 칸만 레드-블랙 트리로

$\text{index} = (h \wedge (h \ggg 16)) \& (n - 1)$ 상위 비트를 섞은 뒤, 하위 비트로 칸을 고른다



정상 상태

해시가 고르면 한 칸에 0~2개

한 칸에 8개가 몰릴 확률

≈ 0.00000006 (1억분의 6)

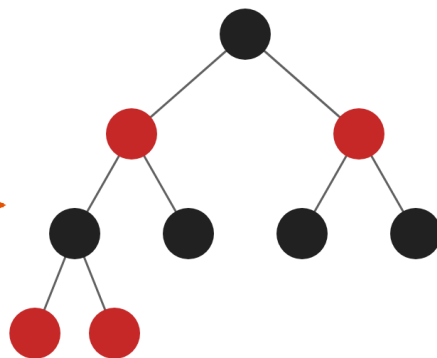
테이블이 64칸 미만이면

트리화 대신 리사이징(2배)부터

같은 칸에 9개
찾기 $O(n)$

트리화

체인 > 8
테이블 ≥ 64



레드-블랙 트리: 최악이 $O(n)$ 에서 $O(\log n)$ 으로
6개 이하로 줄면 다시 리스트 (경계에서 반복 전환 방지)

8

트리로 바꾸는 기준

정상 해시라면 한 칸에 8개 몰릴 확률은 1억분의 6. 넘었다면 비정상 신호다

6

리스트로 되돌리는 기준

8과 간격을 뒤 경계에서 리스트와 트리를 오가는 걸 막는다

64

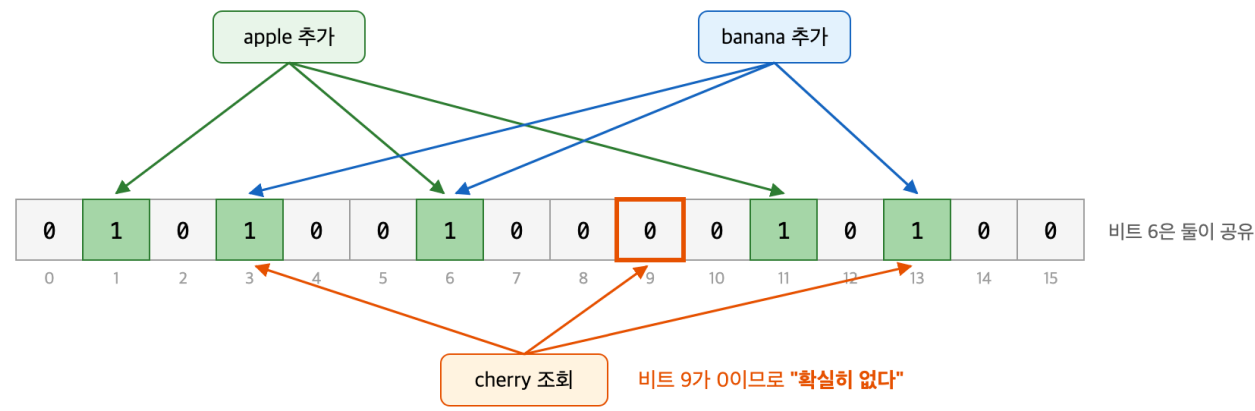
트리화 최소 테이블 크기

작은 테이블의 충돌은 칸 부족 탓. 리사이징이 먼저다

hash flooding (2011): "Aa"와 "BB"의 해시가 같으니 AaAa, AaBB, BBAA, BBBB도 전부 같다. 해시가 같은 키를 대량으로 보내면 한 줄에 쌓여 넣는 데 $O(n^2)$. JDK 8(JEP 180)부터 최악이 $O(\log n)$ 으로 묶였다.

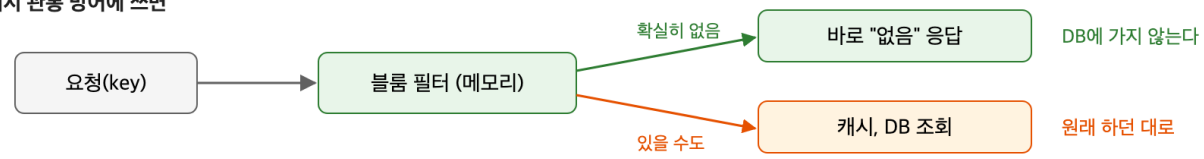
블룸 필터: '확실히 없음'만큼은 메모리에서 답한다

블룸 필터: 비트 배열 하나 + 해시 함수 k개 (그림은 k = 3)



반대로 세 비트가 모두 1이면 "있을 수도 있다". 다른 원소들이 켜 둔 비트와 우연히 겹친 것일 수 있다(거짓 양성).

캐시 관통 방어에 쓰면



거짓 음성 없음

있는데 없다고는 안 한다. 그래서 '없음'이면 DB에 안 가도 된다

거짓 양성 있음

모두 1이면 '있을 수도'. 그때만 원래대로 DB를 조회한다

1%면 원소당 9.6비트

해시 7개. 1억 개도 약 120MB, 키 길이와 무관

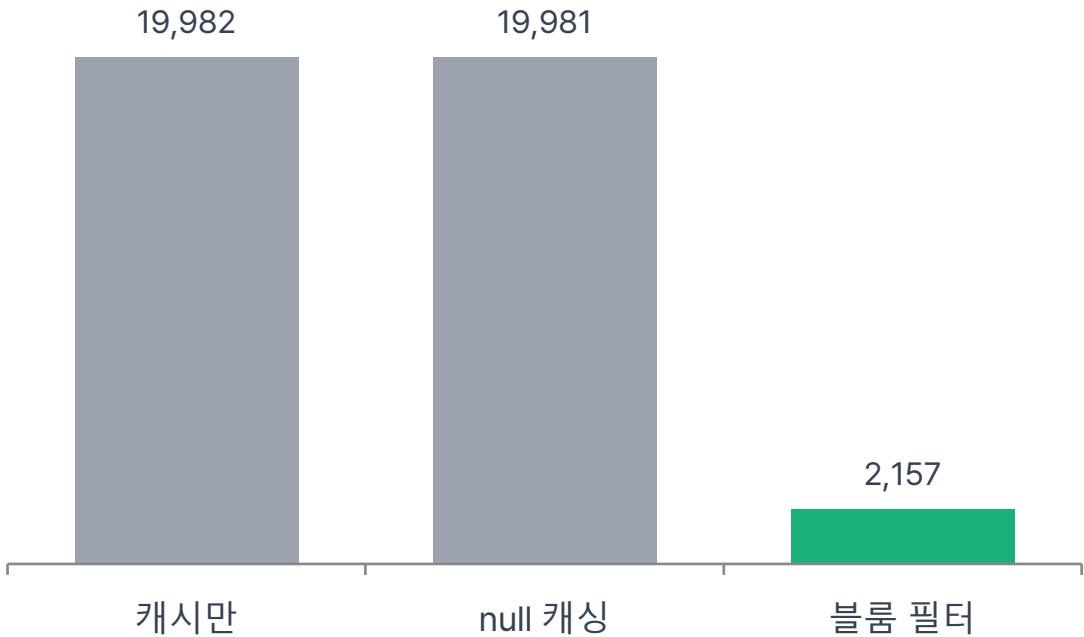
대가

삭제가 안 되고, 예상 원소 수를 넘기면 다시 만들어야 한다

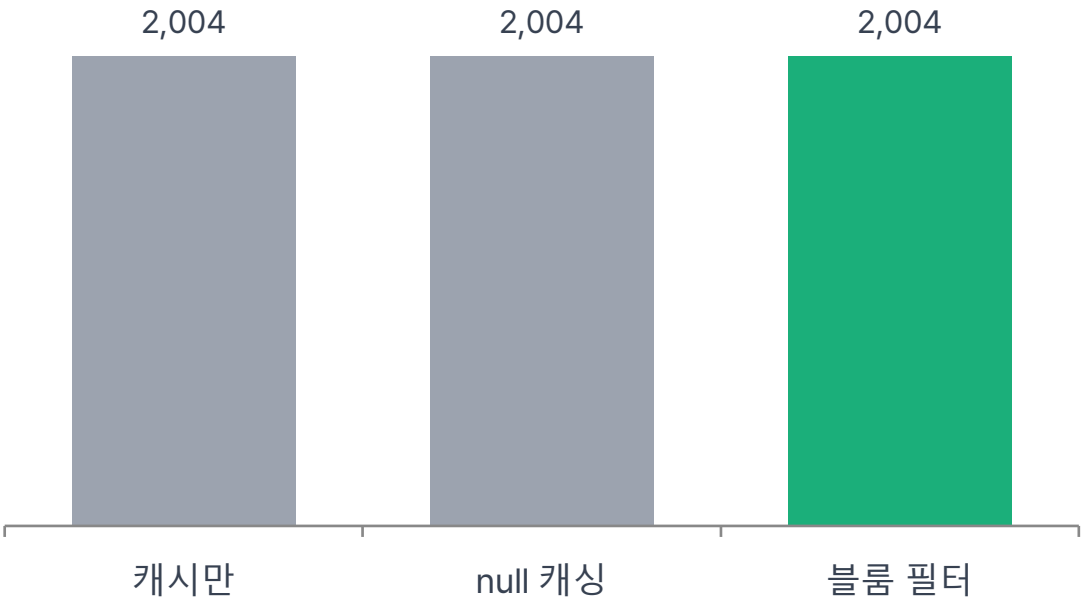
앞에서: 해시는 있는 키를 빨리 찾는다 → 해시를 k개 쓰고 값 대신 비트만 남기면, 없는 키를 적은 메모리로 거를 수 있다

실측: DB 조회 89% 감소, null 캐싱은 무작위 키에 무력

DB 조회 수 (요청 2만 건 중)



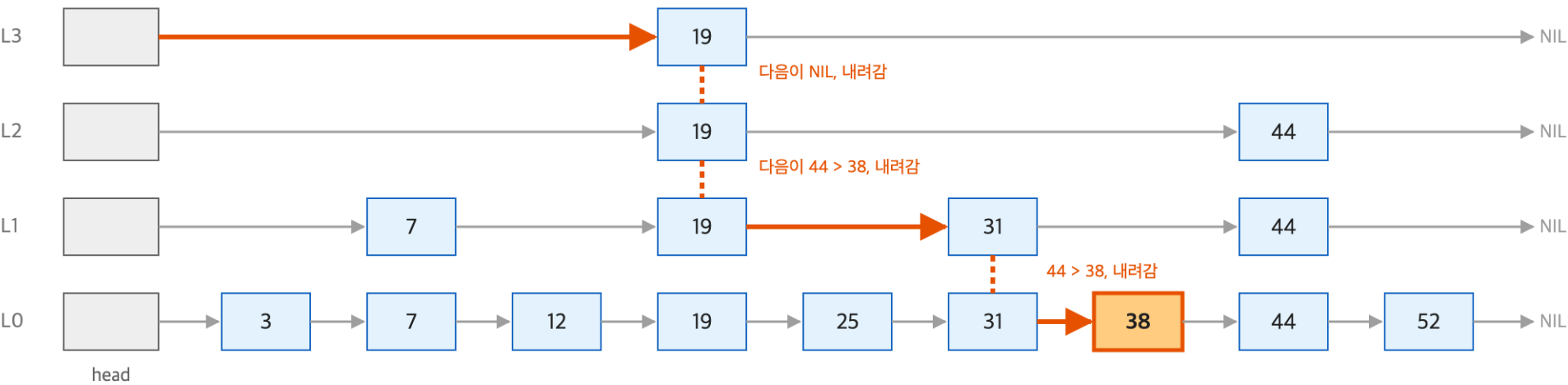
찾은 행: 세 방식 모두 같다 = 틀린 응답 없음



Redis + PostgreSQL 10만 행, 요청의 90%가 매번 다른 없는 키. 남은 2,157건 = 있는 키 2,004 + 거짓 양성 153.
필터 용량 안에서는 설정 1% → 실측 1.01%. 용량을 2배 넘겨 넣으면 15.8%로 뛴다.

스킵리스트: 고속도로가 있는 정렬된 연결 리스트

스킵리스트에서 38 찾기 — 위층에서 크게 건너뛰고, 지나치면 한 층 내려간다



층은 노드를 넣을 때 동전 던지기로 정한다. Redis는 한 층 더 올라갈 확률 1/4, 최대 32층. 거쳐 가는 노드 수의 기댓값이 $O(\log n)$ 이다.
삽입은 해당 층들의 포인터만 바꾸면 끝난다. 균형 트리와 달리 회전(rotation)이 없다.

Redis sorted set의 실제 구조: 원소 128개 이하면 listpack(연속 배열), 넘으면 skiplist(점수 순서) + dict(멤버에서 점수로)

층은 동전 던지기로

Redis는 1/4 확률로 한 층 더, 최대 32층. 회전 없이 기대 $O(\log n)$

범위 조회가 자연스럽다

맨 아래층이 정렬된 리스트. ZRANGE는 옆으로 따라 가면 끝

순위도 $O(\log n)$

포인터마다 건너뛴 노드 수(span)를 저장해 ZRANK를 계산

앞에서: 해시는 순서를 버린다 → 게임 랭킹처럼 점수 순서가 필요하면?

Redis sorted set은 앞의 세 주제를 함께 쓴다

작을 때

128개 이하, 값 64B 이하

listpack

연속 메모리 배열을 $O(n)$ 으로 훑는다. 작으면 포인터를 따라가는 $O(\log n)$ 보다 빠르고 작다

① 캐시 지역성

클 때

멤버 → 점수

dict (해시테이블)

ZSCORE를 $O(1)$ 로 찾는다

② 해시테이블

클 때

점수 순서

skiplist

ZRANGE, ZRANK를 $O(\log n)$ 으로 처리한다

④ 스킵리스트

한 자료구조로 모든 연산을 해결하지 않고, 크기와 연산마다 맞는 구조를 고른다.

디스크에서는 트리의 높이가 곧 디스크에 가는 횟수다

27번

이진 트리를 디스크에

1억 개면 높이 $\log_2(1\text{억}) \approx 27$.

노드마다 디스크 어딘가에 있으니 랜덤 읽기 27번.

노드 하나 읽으려고 8KB 페이지를 통째로 읽고 버린다.

3번

B-tree: 노드 하나 = 페이지 하나

8KB 페이지에 키 약 500개 → 가지도 500개.

$500^3 \approx 1\text{억 } 2,500\text{만}$. 1억 행도 높이 3~4.

위쪽 층은 늘 버퍼 풀(메모리)에 있어 실제 디스크 읽기는 1~2번.

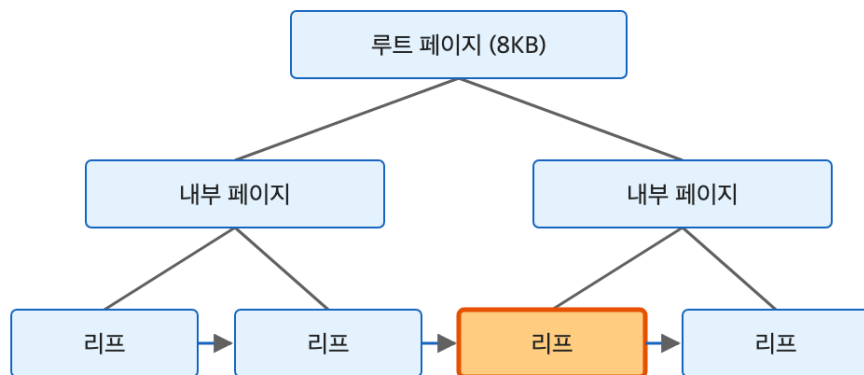
대가는 쓰기: 키마다 들어갈 리프가 정해져 있어 그 자리를 고쳐 쓴다. 100바이트를 고쳐도 8KB 페이지를 쓰고, 키가 무작위면 매년 다른 페이지라 랜덤 쓰기.

실무: InnoDB에 AUTO_INCREMENT 키는 맨 오른쪽 리프에만 쌓이지만, 무작위 UUID는 아무 리프에나 들어가 랜덤 쓰기와 페이지 분할이 잦다.

앞에서: 정렬된 데이터를 메모리에 두는 법(스킵리스트) → 데이터가 메모리를 넘으면 디스크에 어떻게 두나? 전송 단위만 64B에서 8KB로 커진다

디스크 위의 두 선택: 제자리 덮어쓰기 vs 모았다가 순차로

B-tree: 제자리에 덮어쓴다



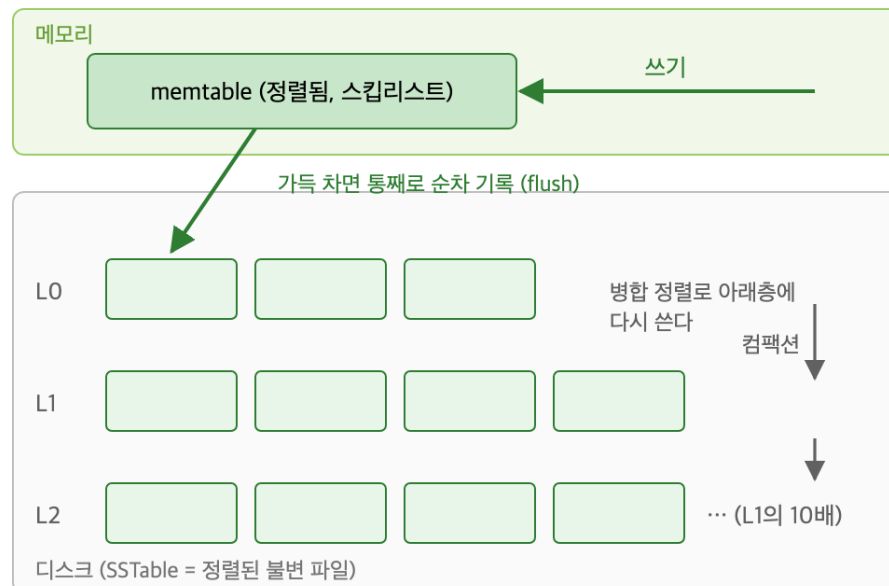
1행 수정이어도 이 8KB 페이지를 그 자리에 다시 쓴다

- 노드 1개 = 디스크 페이지 1개, 키가 수백 개 들어간다
- 팬아웃이 커서 수억 행이어도 높이는 3~4
- 위쪽 페이지는 늘 메모리에 있어, 실제 디스크 읽기는 1~2번
- 리프끼리 연결되어 있어 범위 조회는 순차 읽기

강점 읽기가 빠르고 예측 가능하다

대가 랜덤 쓰기, 페이지 단위로 커지는 쓰기 양

LSM 트리: 모았다가 순차로 쓴다



- 읽기는 memtable, L0, L1, L2 ... 를 차례로 확인한다
- SSTable마다 bloom 필터를 뒤서 키가 없는 파일은 건너뛰다

강점 디스크 쓰기가 항상 순차다

대가 여러 곳을 뒤지는 읽기, 컴팩션의 반복 재쓰기

발상의 전환: 이미 쓴 파일은 절대 고치지 말자. 새로 들어온 것은 메모리에 모아 정렬한 뒤 새 파일로 한꺼번에 쓰자

LSM 따라가기: 쓰기 한 번, 읽기 한 번

쓰기: 42 = D

1

memtable(스킵리스트)에 넣는다

메모리라 빠르다. 수정은 덮어쓰고, 삭제는 tombstone(삭제 표시)을 넣는다. WAL에도 기록

2

차면 SSTable 파일로 내려쓴다 (flush)

이미 정렬돼 있으니 앞에서부터 쭉. 완전한 순차 쓰기. 기존 파일은 건드리지 않음

3

파일이 쌓이면 컴팩션

여러 파일을 병합 정렬해 아래 층에 새로 쓴다. 같은 키는 최신만 남기고 옛 버전·삭제분 정리

4

대가: 같은 데이터를 층마다 다시 쓴다

쓰기 총량은 B-tree보다 클 수 있다. 줄인 건 총량이 아니라 랜덤 쓰기

읽기: 42는?

1

memtable을 먼저 본다

있으면 그게 최신. 끝

2

L0 파일들을 최신부터 본다

같은 키의 옛 버전이 아래에 있을 수 있어, 위에서 먼저 만난 값이 답

3

L1, L2 ...는 층마다 파일 하나씩

한 층 안에서는 파일끼리 키 범위가 안 겹친다

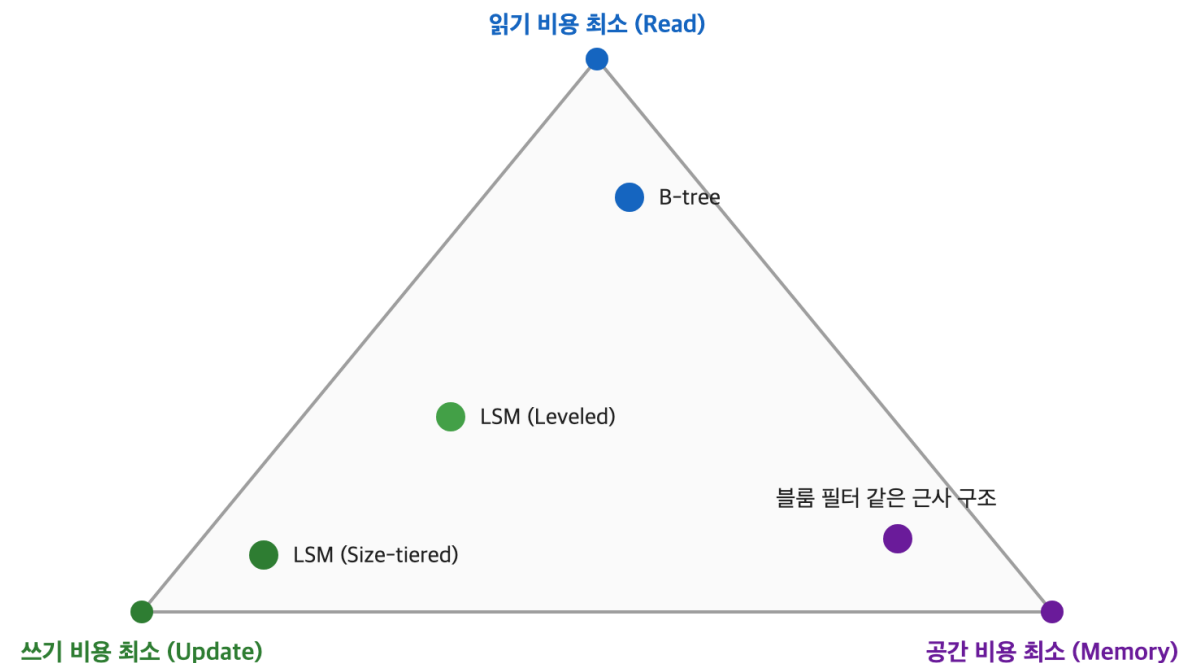
4

파일마다 블룸 필터로 건너뛴다

"이 파일에 42는 확실히 없음"이면 디스크에서 안 읽는다. 1부 ③이 여기서 쓰인다

셋 다 얻을 수는 없다: 어떤 증폭을 받아들일까

RUM 추측: 읽기·쓰기·공간 비용을 동시에 최소화할 수는 없다



꼭짓점에 가까울수록 그 비용이 작다. 한 꼭짓점에 다가가면 나머지 둘에서는 멀어진다. 위치는 개념적 배치다.

읽기 증폭

논리적 읽기 1번에 실제로 읽는 페이지·파일 수

쓰기 증폭

1바이트 쓸 때 디스크에 실제로 쓰는 양. LSM은 컴팩션 때문에 오히려 클 수 있다. LSM이 줄인 건 총량이 아니라 랜덤 쓰기

공간 증폭

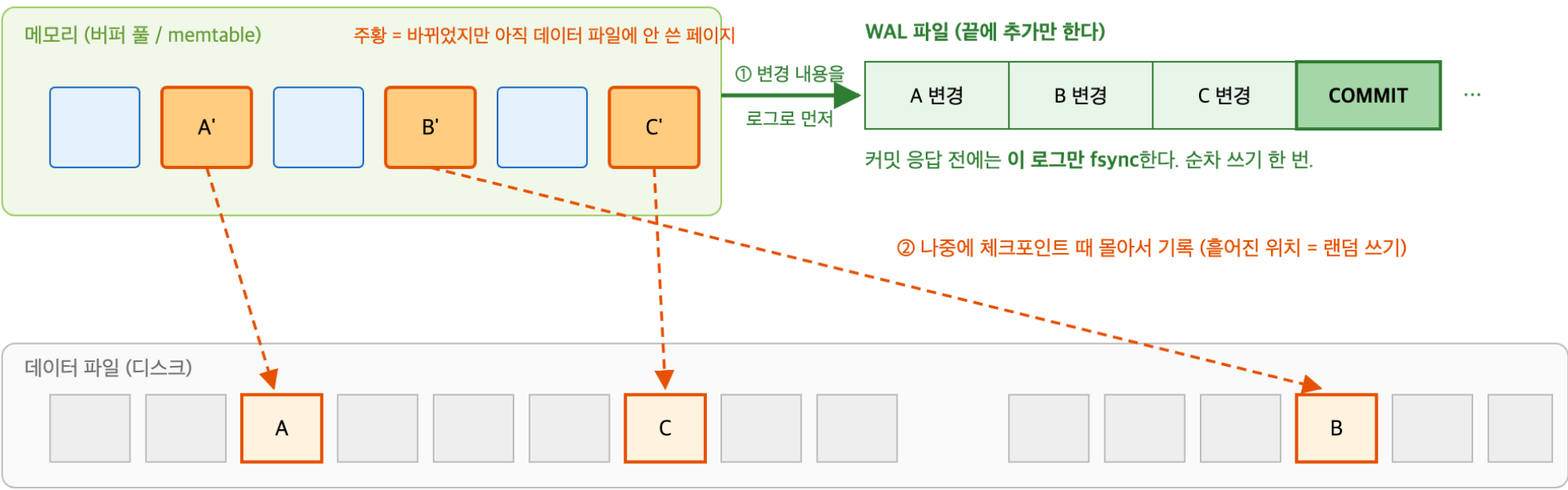
실제 데이터 대비 디스크 사용량

워크로드에 고른다

읽기 많음, 트랜잭션, 범위 조회: B-tree 계열 RDB
로그·이벤트·시계열처럼 쓰기 폭주: LSM 검토

WAL: 커밋은 로그 한 줄, 페이지는 나중에

WAL: 데이터 페이지보다 로그를 먼저 쓴다



③ 크래시가 나면 데이터 파일은 마지막 체크포인트 시점 상태다. 그 뒤의 변경은 WAL을 재실행(redo)해서 되살린다.

그래서 커밋할 때마다 흩어진 페이지를 쓸 필요가 없다. 랜덤 쓰기 여러 번이 순차 쓰기 한 번으로 바뀐다.

PostgreSQL에서는 바뀐 버퍼 풀 페이지를, RocksDB와 Cassandra에서는 아직 flush 안 된 memtable을 지킨다.

앞에서: B-tree도 LSM도 변경을 일단 메모리에 들고 있다 → 그 상태에서 서버가 죽으면?

여섯 개의 자료구조, 한 장으로

자료구조	푸는 문제	대가	실무 포인트
① ArrayList	연속 메모리로 캐시를 꽉 채운다	중간 이동 $O(n)$	리스트의 기본값
② HashMap	평균 $O(1)$ 조회	리사이징 순간 $O(n)$	크기를 알면 초기 용량 지정
③ 블룸 필터	없는 키를 메모리에서 차단	거짓 양성, 삭제 불가	캐시 관통 방어 (DB 조회 89%↓)
④ 스킵리스트	정렬·범위·순위를 $O(\log n)$	포인터 메모리	Redis sorted set (+ dict, listpack)
⑤ B-tree	페이지 단위로 빠른 읽기	랜덤 쓰기	읽기 위주 RDB (InnoDB)
⑤ LSM 트리	쓰기를 순차로	읽기 증폭, 컴팩션	쓰기 폭주 (RocksDB, Cassandra)
⑥ WAL	빠른 커밋 + 내구성	로그 양, 체크포인트 비용	거의 모든 DB의 기본

언제 쓰면 안 되나

LinkedList

이터레이터로 위치를 주고 삽입·삭제를 반복할 때만 의미가 있다.
양 끝 용도는 ArrayDeque가 대부분 더 빠르다

블룸 필터

같은 없는 키가 반복되면 null 캐싱으로 충분하다. 키가 매번 다른
무작위 공격일 때 필요하다

LSM 트리

쓰기는 순차지만 컴팩션이 몰리면 지연이 된다. 읽기 위주 트랜잭션
서비스엔 B-tree가 기본값이다

HashMap 리사이징

평균은 $O(1)$ 이어도 리사이징이 일어나는 put 한 번은 $O(n)$ 이다.
큰 맵을 키운다면 초기 용량을 지정한다